Mar & Kuku Production Studio

Operator handbook and first manual test

Edition: 23 October 2026 | Application version: 0.1.0 | Reviewed baseline: ff39fb7

This is your guide to operating MK Studio yourself: what it is, how to open it, what the controls do, what you contribute, and how to judge whether a production run actually succeeded.

The short answer: MK Studio is a local Python web application with its own browser-based chat and dashboard. You start its server on your Mac, then use it in a browser. It also includes a command-line tool called `mk-studio`. It is currently a development MVP, with useful working components and important integration gaps.

It is not currently distributed as a DMG, a VS Code extension, or a public hosted website. There is no `npm run dev` step. VS Code and Antigravity are development workspaces; they are not the intended daily production interface.

Before your first test: Opening the dashboard and listening to existing recordings are reasonable first checks. Do not use the current CH02 approval/trim or resume controls as a production acceptance test until the defects on pages 5 and 12 are fixed.

Find what you need

Page	Topic
2	How the app, browser, AI services and Mac fit together
3	Start, stop and restart the app
4	Dashboard and button reference
5	Current readiness: working components and limitations
6	Your first manual test, step by step
7	Your role and the intended production journey
8	Browser accounts, audio generation and review
9	Where files live and what to back up
10	Complete CLI command reference
11	Troubleshooting and daily operating habits
12	Next milestone, acceptance criteria and evidence

Scope of this edition: Documentation update only; this review did not change application code or media. The old PDF is preserved under `scratch/handbook-review-20261023/MVP_Guide_before_operator_update.pdf`. This edition supersedes its startup instructions and readiness claims.

Concurrent development: Uncommitted changes appeared during preparation in configuration, workflow routing and production export. They are not accepted fixes. Read the readiness findings as the ff39fb7 baseline; confirm the final code version and test results before production use.

1. What you are operating

Four parts work together

Part	Its job
Studio dashboard	Your local chat, workflow progress, saved messages, job history and audio players. Open it at http://127.0.0.1:8767/.
Python server	Interprets supported instructions, calls workflow functions, tracks files and decisions, and coordinates other tools.
Managed Chrome	A separate Chrome profile that Studio controls through Chrome DevTools Protocol (CDP). It visits your AI services and handles supported browser tasks.
Local production tools	FFmpeg processes media; Palmier Pro holds editable video projects; SQLite stores application state. Mac automation supports some desktop actions.

Typical direction of work: You -> Studio chat -> workflow engine -> browser services and local tools -> saved files and receipts -> Studio shows the results -> you review.

Does Studio have its own AI?

Studio has its own application logic and provider connections. It does not contain its own trained language model. Its intended team uses ChatGPT as lead planner/reviewer, Gemini as a second reviewer, and Manus for specialist work such as transcription. Those services run through supported browser adapters. Codex CLI and an optional OpenAI API provider also exist, but they are separate execution choices.

The current chat dispatcher uses keyword rules for many actions. It is not a fully general conversational producer. Some ordinary sentences can trigger an action unexpectedly; see the CH02 approval issue on page 5.

Where should you work?

Use **Studio in your browser** for the intended production experience. Use VS Code or Antigravity when developing, troubleshooting or discussing a code change. A coding agent should not need to run temporary scripts for a workflow that Studio claims to support autonomously.

Platform today: This installation targets macOS. Paths, Keychain, AppleScript, local speech and editor integration include Mac-specific assumptions. Your HP Windows machine is not yet a verified Studio installation. Linux and Windows support require development and validation; copying this folder alone is not a supported migration.

Local versus online: The server, database and saved media live on your Mac. Browser AI work still needs internet access, an eligible account and available service credits. Keeping Studio running does not itself mean an AI generation is taking place.

2. Start, stop and restart

Start on this existing Mac installation

Open Terminal, or the Terminal panel in VS Code. Run:

```
cd /Users/yonassayfu/Projects/mar-kuku-production-studio
.venv/bin/mk-studio serve --host 127.0.0.1 --port 8767
```

Keep that terminal open. In your browser, visit:

```
http://127.0.0.1:8767/
```

Or open a second terminal tab and run:

```
open http://127.0.0.1:8767/
```

The `.venv/bin/` prefix selects the project's installed Python environment. You do not need to activate it first. If you prefer activation, use `source .venv/bin/activate`, then `mk-studio serve --host 127.0.0.1 --port 8767`.

Port note: The CLI defaults to 8765 if you omit `--port`. This handbook consistently uses 8767, matching the latest handoff. The browser URL must match the port used to launch the server.

Does it start when the Mac starts?

No Studio startup service was found in the inspected project, user/system LaunchAgents or relevant loaded launch services. Treat startup as manual. At inspection time, nothing was listening on port 8767. A previous message saying the app was running does not mean it remains running after a shutdown or restart.

The current environment can import the server dependencies. This is a guide for the existing installation, not a verified fresh-machine installer. Do not rebuild the environment simply to open the app.

Three different kinds of stop

Action	Effect
Studio Stop button	Requests cooperative cancellation of work. An already submitted remote task or blocking operation may continue until the next checkpoint.
Close browser tab	Closes your view. The server may keep running in its terminal.
Ctrl+C in server terminal	Shuts down the local server. Stop active work first and wait for its status where possible.

Restart with the same launch command and same project/output root. Messages are stored on disk, but do not assume every workflow resumes correctly: CH02 resume currently resets decision status. After a crash, inspect remote tasks before retrying a generation.

3. Reading the dashboard

The chat is your instruction area. Workflow cards show stages, while job history and browser/Mac activity help explain what happened. Audio attachments can be played inside message bubbles. A green badge must be checked against the actual output and its receipt.

Control	Current meaning and operator guidance
Resume CH02 Audio	Imports/registers existing CH02 candidates and reports historical work. It resets decision states; defer this control until fixed. It is not a fresh-generation demonstration.
Approve & Trim 4s	Submits a fixed approval-and-repair message. Current routing and trim behavior have defects. Do not use for production approval yet.
1-4: 3-Agent Script	Starts the ChatGPT/Gemini/Manus script consultation workflow. It can submit real requests and consume credits. Confirm the intended episode first.
6: Assets	Runs the asset stage for the active episode. Existing paths include local test voices/props; this button alone does not establish the requested Flow Music workflow.
7: Palmier	Runs episode assembly. Requires compatible inputs and a working Palmier connection. It changes a scratch editor project.
8: MP4 Export	Reviewed baseline defaults to FFmpeg preview. An unverified edit changes the default to native with fallback. Check the receipt; an MP4 alone does not prove native rendering.
9: YouTube	Stages the package and opens/checks YouTube Studio. This button is not evidence of a completed upload or publication.
Full Pipeline (6-9)	Chains the current production stages. It is not yet the accepted complete browser-audio-to-final-video workflow. Defer for your first test.
Resume	Resumes the general script workflow according to its checkpoints. This is different from Resume CH02 Audio.
Verify CH01	Runs technical proof checks. It may write scratch reports or prepare missing proof inputs. It does not approve creative quality.
Inspect Mac	Inspects supported desktop/permission information. It does not establish unrestricted automation of every app.
Sandbox Fixture	Local browser test/demo. It is not ChatGPT, Manus or another external provider.

Browser and provider controls

Owner Takeover pauses browser mutations so you can interact with the managed Chrome session, including login. **Resume Auto** releases that mode; it does not necessarily resume a stopped workflow. **Refresh Preview** captures the browser view. **Refresh** reloads displayed state.

The provider selector affects ordinary chat requests. Explicit workflow actions can choose their own providers. Selecting ChatGPT does not prove that every subsequent production step uses ChatGPT or browser mode.

4. Where development stands

Assessment of ff39fb7: a development MVP with technical proofs and partial production integration. Inspecting the interface and existing artifacts is the first test. An unattended idea-to-upload workflow has not been demonstrated by this review. New uncommitted edits require their own validation.

Area	Evidence and practical limit
Local app and chat	Python CLI and web server are implemented; saved messages, workflow state and audio endpoints exist. Startup help and dependency imports were checked for this edition.
Multi-agent scripts	Browser adapters and persistent consultation code exist; earlier reports describe live runs. No new paid consultation was run for this handbook.
Flow Music and Manus	Download, generation and file-upload code exists, with historical CH02 artifacts. Some historical steps were driven by developer scripts. Importing those files does not prove full Studio orchestration.
CH01 proof	Existing proof commands/projects and prior verification reports exist. Full editor verification was not rerun for this documentation update.
CH02 audio review	Existing files and review UI are present. Approval, duplicate-candidate and resume issues prevent accepting the current flow as reliable.
Video and YouTube	Native project support and preview rendering exist. Baseline UI export defaults to preview; an unverified edit now attempts native first. UI YouTube stages a package. Final native export and upload need a verified complete run.

Current issues that affect your test

Candidates A and B contain the same decoded audio. Both are 84.512 seconds long and have the same decoded PCM hash in this inspection. Different filenames or whole-file hashes are not proof of different performances. A claim that one sounds better than the other is not supported by these files.

Baseline approval routing is too broad. A sentence with approval/trim plus audio/script keywords can trigger repair even when rejecting it. An uncommitted edit adds rejection checks, but end-to-end handling is unverified. Do not test negative approvals in production chat. Accept-as-is also needs separate handling.

The trim claim does not match the edit. The code removes 3.6 seconds, producing an 80.912-second revision. Its message claims a 10.6-second gap became 4 seconds, which would require removing 6.6 seconds. Actual vocal boundaries must be verified before choosing corrected cuts.

CH02 resume is not a harmless status refresh. It unconditionally restores awaiting-owner states. The repair action also reuses the same revision filename. These need correction before repeated decisions or reliable restart testing.

Automated tests are not listening approval. A current integration test uses the real default runtime folder and submits an approval message. Do not run the entire test suite against your working production state until that test is isolated.

5. Your first manual test

Today: inspect, listen and record feedback

1. Start Studio using page 3. Open the dashboard at port 8767. Confirm the title and that the page loads. If it does not, use page 11.

2. Read the displayed conversation and workflow cards. Note the episode/chapter name. The default conversation can still refer to CH01 while CH02 material appears; do not infer the active production target from the title alone.

3. Play existing audio attachments if they are already visible. Listen to each original from beginning to end. If players are absent, open the files directly using the commands below. You do not need to click Resume CH02 to listen.

4. Compare the recording with the approved chapter script. Record wrong or missing words, long pauses, clipped syllables, distorted joins and unclear counting. Write feedback in a separate note for now; avoid approval/trim wording in the current Studio chat.

5. Refresh the browser page. Existing saved messages should return. This tests display persistence, not resume correctness. Do not interpret an existing approved status as your own approval without checking its origin.

6. Stop the server with Ctrl+C when finished. Save your notes and the exact filenames you listened to. Report any error with its time, button/action and visible message.

Open the recordings without changing workflow state

From the project terminal:

```
cd /Users/yonassayfu/Projects/mar-kuku-production-studio
open scratch/ch01-proof/episodes/EP02_CH02_AUDIO/00_ORIGINAL/EP02_CH02_Candidate_A.wav
open scratch/ch01-proof/episodes/EP02_CH02_AUDIO/00_ORIGINAL/EP02_CH02_Candidate_B.wav
```

The revision can also be auditioned, but it is an unaccepted review file:

```
open scratch/ch01-proof/episodes/EP02_CH02_AUDIO/EP02_CH02_Candidate_B_v002_EDITED.wav
```

What counts as passing this first check? The app opens; you understand the displayed chapter; audio plays; you can locate the source files; saved messages survive refresh. These checks do not establish correct automatic trimming, generation, native rendering or uploading.

After the fixes: The next acceptance test should start with one Studio instruction, visibly run the browser chain, produce distinct new candidates, obtain a traceable review, apply only your intended decision, and survive restart without repeating paid steps. Page 12 defines that gate.

6. Your role in making a video

You are the producer and creative decision-maker. Studio should handle repetitive transfers, file organization, timing calculations and supported tool actions. Your contribution is to set direction and judge the result at meaningful points.

Stage	What you contribute	What Studio should return
Episode brief	Topic, age group, learning goal, language, duration and which episode/chapter to work on.	A clearly identified project and proposed plan.
Script	Review wording, teaching accuracy, character behavior and response pauses.	A versioned script and a record of the accepted version.
Audio reference	Choose the previous approved performance/style to reference.	A generation prompt tied to that reference and the new script.
Candidate review	Listen, compare voices/music and identify pronunciation or wording problems.	Actual candidate audio, transcripts, comparison and proposed repairs.
Audio decision	Accept an exact version, request regeneration, or approve a specific repair. Wording changes and timing edits are separate decisions.	A new revision with provenance, revised timestamps and a listening checkpoint.
Visual edit	Review character consistency, poses, teaching objects, timing and motion.	Editable project plus an accurately labeled preview/export.
Delivery	Review the final video, title, thumbnail, channel and visibility.	Upload receipt and actual channel/video result when upload is implemented and verified.

Intended full workflow

Approved brief -> browser AI consultation -> saved script -> Flow Music reference remix with new script -> download generated candidates -> Manus transcripts -> ChatGPT comparison using the actual available inputs -> explicit decision -> verified audio repair if needed -> refreshed cues -> Palmier assembly -> export and QA -> YouTube delivery.

This is the product goal. Current code supports portions of it; the entire chain must still be accepted as one Studio-driven run.

Give feedback that can be acted on

Use a filename/version, a time range, the observed problem and the intended change. For example: "Candidate B, around 00:30: the word Red sounds stretched. Keep the approved wording and give me a replacement take for review." Save this as a note until the current dispatcher is fixed.

A request to repair audio is not approval of the repaired result. A transcription is not proof of silence. A technical pass is not approval of teaching quality. If you delegate a decision to ChatGPT, define the scope explicitly; a provider review should not automatically become unrestricted approval of the video or publication.

7. Browser accounts and audio work

Two browser roles

Your dashboard tab shows Studio. The managed Chrome window performs supported work on external services. Studio launches or connects to Chrome on local CDP port 9222 with its dedicated profile under `scratch/ch01-proof/browser-profile`.

That profile is separate from your normal Chrome profile. Being logged into ChatGPT in your everyday browser does not establish that Studio's browser is logged in. Sessions may persist across restarts, but can expire or require verification.

For login: use Owner Takeover, work in the managed Chrome window, complete the service's login/verification, then use Resume Auto. Check the visible account and service before beginning production. Do not have Agy, Studio and yourself simultaneously drive the same browser tab.

Your intended music workflow

The service discussed in this project is Google Flow Music, reached at `https://www.flowmusic.app/`. The desired process is more than downloading an old song: open the chosen reference, use its Remix/Start session flow, submit the newly approved script/prompt, wait for generation, and download each actual new result.

Studio must identify which generation and candidate each download belongs to. If two results are offered, save them separately and verify that they contain different audio before comparing them. The current CH02 files fail that distinction.

Each candidate should go to Manus with the audio attached. Save the returned transcript and timestamps alongside the audio. Send the relevant audio and transcript to ChatGPT when the active account supports attachments. If only text was submitted, the review must say so. A text review cannot establish that the model listened to the WAV.

What “edited audio” must mean

An editing recommendation is not an edited WAV. If the browser session cannot return an audio file, Studio can apply a clearly specified local FFmpeg edit and save a new version. It must preserve words, map source times to edited times, and verify the result. Trimming cannot change “Have a look” into “And look.” That requires an accepted script change or a new/replacement performance.

The response hold must contain no vocals for the required duration; it may still contain intended background music. Its location comes from the actual chapter script and performance, not a universal CH01 timestamp.

Credits: Generation, consultation and some transcription requests can consume service credits. Reading local files and replaying audio does not submit another generation. After a timeout, check the service's task/history before retrying; a local timeout does not prove the remote job failed.

8. Files, backups and recovery

All relative paths below start at the application project folder.

Location	What it contains
src/mk_studio/	Application source code.
docs/implementation/	Guides, plans and implementation reports.
scratch/ch01-proof/state.db	SQLite state: conversations, jobs and artifact records.
scratch/ch01-proof/episodes/	Episode outputs, saved scripts, manifests, receipts and media.
episodes/EP02_CH02_AUDIO/ under that folder	Current CH02 candidate, transcript, review and revision files.
scratch/ch01-proof/browser-profile/	Managed Chrome's local profile and sessions.
scratch/ch01-proof/browser-captures/	Browser screenshots captured by automation.

Approved production library:

```
/Users/yonassayfu/Desktop/DoroNesh_2D_Animation/
```

The intended file policy is to read approved production inputs and write generated work into scratch. “Scratch” is the folder name, not permission to throw it away: it currently contains real workflow state and media.

GitHub is not a complete production backup

The repository ignores scratch, databases, browser profiles, WAVs, MP4s and Palmier bundles. A successful git push backs up committed source/documents, not all of your production work. Uncommitted changes are also not in that push.

For a complete local backup, first stop active jobs and the Studio server; close managed Chrome if copying its profile. Copy the project including scratch and separately copy the production library to a backup drive or your normal Mac backup system. Keep SQLite database sidecar files together if present. A live, partial database copy is not a reliable recovery plan.

Do not delete the browser profile to “reset” Studio: that removes saved sessions. Do not delete state.db as routine troubleshooting. Restore from a known backup when necessary and verify that media paths still exist.

Understanding a receipt

A manifest records stages and linked artifacts. A SHA-256 is a file fingerprint; it helps detect changes, but does not prove audio quality, script correctness or distinct performances. A receipt should identify the script/version, input audio, output path, renderer/provider, actual result and any approval evidence.

Read `is_preview_render` and renderer details before calling an MP4 a final native export. Check a real video ID/URL and visibility before calling a YouTube stage an upload. A “completed” label is meaningful only for the specific operation recorded.

9. Command reference

Run commands from the project folder. The startup commands on page 3 are all you normally need to open the existing app. The other commands are for diagnostics or deliberate production/developer work.

Help and environment checks

```
.venv/bin/mk-studio --help
.venv/bin/mk-studio serve --help
.venv/bin/mk-studio doctor
```

`doctor` reports paths, tool availability, database tables and Palmier connectivity. It may initialize local state. Read the report fields: its exit code alone is not proof that every dependency is available.

CH01 proof tools

Command after .venv/bin/mk-studio	Purpose / side effects
bind-ch01	Binds pinned CH01 inputs and reproduces audio into scratch; records artifacts/state.
compile-ch01	Builds scene/registration/timeline data; can bind inputs if absent.
assemble-ch01	Changes the scratch Palmier project, verifies reopen and requests export.
assemble-ch01 --skip-export	Performs assembly/reopen without video export.
assemble-ch01 --project PATH	Selects a scratch editor project. Replace PATH with its absolute path.
verify-prep-ch01	Checks offline proof preparation; may prepare missing inputs/write reports.
verify-ch01	Checks the full technical proof contract; may prepare missing data/write reports. It is not a listening approval.

Additional CLI features: developer-assisted use

```
mk-studio produce approve --brief /absolute/brief.json
mk-studio produce prepare --brief /absolute/brief.json
mk-studio produce voice --brief /absolute/brief.json
mk-studio produce render --brief /absolute/brief.json
mk-studio upload-youtube --video /absolute/video.mp4 \
  --metadata /absolute/metadata.json \
  --receipt /absolute/scratch/new-upload-receipt.json
```

These are syntax templates, not copy-and-run examples: replace paths with validated inputs and use `.venv/bin/mk-studio` unless the environment is activated. `approve` records a decision; `voice` and `render` create media; `upload-youtube` performs real browser upload work. Do not use them for the first inspection test. CLI private-by-default metadata does not itself verify a successful private upload.

Optional API setup: `.venv/bin/mk-studio set-secret openai_api` prompts for a key and stores it in macOS Keychain. Browser-mode production does not require that API key. `--output-root /absolute/scratch/path` is a global option placed before a subcommand; changing it creates a different state/profile location. Do not change it casually.

10. Troubleshooting and daily habits

Symptom	What to do
Browser cannot connect	Start the server and check the terminal error. Confirm you used 8767 in both command and URL. Opening a saved HTML guide does not start Studio.
Address already in use	A process is using the port. Identify it with the command below. Reuse the existing Studio instance if appropriate; do not start competing instances or kill unrelated processes.
mk-studio not found	Use `.venv/bin/mk-studio` from the project directory. If the file is missing, ask the developer to restore the environment.
Module/import error	Capture the exact error. The app needs Python 3.12 and its dependencies. A fresh installation needs dependency validation; avoid repairing it with random package installs.
Provider asks for login	Use Owner Takeover in the managed Chrome window. Complete login, then Resume Auto. Check the correct account/profile.
Browser workflow stalls	Inspect the actual service page for a task still running, quota, upload or login prompt. Avoid repeatedly submitting the same request.
Audio player fails	Confirm the file exists, the server is running and the path belongs to the current output root. Open the WAV directly as a diagnostic.
Palmier unavailable	Open Palmier Pro and run doctor. Its configured MCP address is http://127.0.0.1:19789/mcp. Do not assume launching the app guarantees the MCP endpoint is ready.
“Completed” but output wrong	Save the receipt, filename and error. Check renderer, real audio content, script version and upload result. Do not rely on the badge alone.

Useful terminal checks:

```
lsof -nP -iTCP:8767 -sTCP:LISTEN
curl -I http://127.0.0.1:8767/
open -a /Applications/PalmierPro.app
git status --short --branch
git log -1 --oneline
```

`curl` checks HTTP reachability; it does not prove production readiness. Git commands show code state, not whether external AI accounts or the editor are working.

Daily habits: Run one Studio server and one operator-controlled workflow at a time. Keep the Mac awake during active browser/editor work. Confirm the active episode before expensive steps. Let automation control its browser while it is working; use takeover before manual interaction. Save listening notes with timestamps and filenames.

When reporting a problem: Include the code commit, action or exact message, episode/chapter, time, expected result, actual result and relevant output path. Keep your production inputs intact while the developer investigates.

11. What must happen next

The next useful milestone is a reliable single-chapter run operated entirely from Studio, including the real browser audio workflow and correct decisions. Expanding to more agents or more chapters should follow that acceptance test.

Acceptance gate before full production testing

1. Fix chat intent handling so rejection, accept-as-is, trim-only, wording changes and regeneration remain distinct. A negative sentence must not authorize repair.

2. Identify and download genuinely distinct generated candidates. Verify decoded audio content as well as filenames and file hashes.

3. Derive repairs from verified vocal boundaries and the chapter script. Save a new version, remap timestamps and validate the requested no-vocal hold. Do not present the existing 80.912-second revision as proof of a corrected 4-second pause.

4. Preserve decisions and completed steps on restart/resume. Isolate tests in temporary output roots so tests cannot create owner approvals or overwrite working media.

5. Demonstrate the full browser chain from one Studio instruction with no temporary developer scripts. Reuse verified completed work; track remote submissions before retrying.

6. Bind the approved script/audio to the correct Palmier project. Distinguish native export from preview and verify the actual resulting video before advancing to upload.

7. Demonstrate a deliberate private upload with a real channel result before treating upload as complete. Public publication is a separate final decision.

Longer roadmap

The approved Episode 002 scope is **CH01-CH09**, not seven chapters: welcome; Red/Yellow/Blue; Green/Orange/Purple; first-group matching; second-group matching; colour detective; song; recall Red/Yellow/Orange; celebration/goodbye. Expand the proven chapter engine to that scope, then complete full-episode joins and QA. Broader desktop control, additional providers, packaging/startup convenience and Windows/Linux portability remain separate development work.

Evidence behind this edition

Inspected on 23 October 2026 at local commit `ff39fb7`: `pyproject.toml`, CLI help/source, `studio/server.py`, `studio/workflow.py`, `production/pipeline.py`, `browser/controller.py`, `config.py`, `.gitignore`, CH02 integration test, the approved requirements and existing guide. Paths are relative to `src/mk_studio/` where applicable.

Read-only audio probing found both CH02 originals at 84.512 seconds with identical decoded PCM SHA-256: `5333d9df100ec96ded70f4099ffbfa79ff9c583fc714635fc5657876e9918cea`. The edited revision probes at 80.912 seconds. No generation, repair, editor assembly, upload or full test suite was run for this handbook.

Use this edition with its code snapshot. If Agy changes the app after ff39fb7, ask for a documented live acceptance run and update this guide with the new evidence. A newer summary alone is not the same as a verified result.